Presentation for use with the textbook Data Structures and Algorithms in Java, 6th edition, by M. T. Goodrich, R. Tamassia, and M. H. Goldwasser, Wiley, 2014

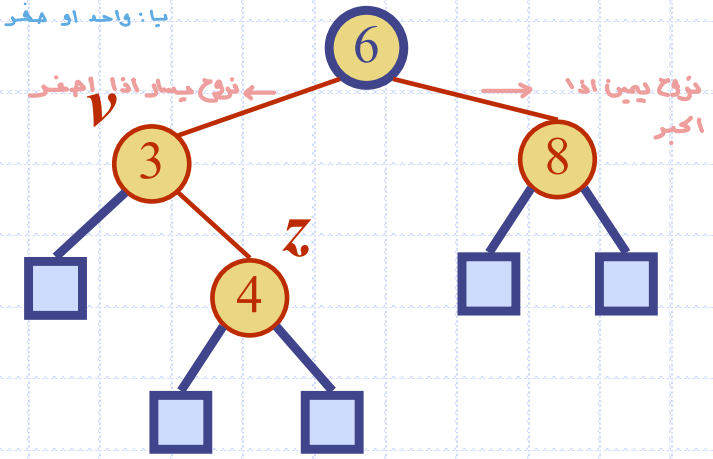
AVL Trees

binary Search tree + balanced

متوازنة

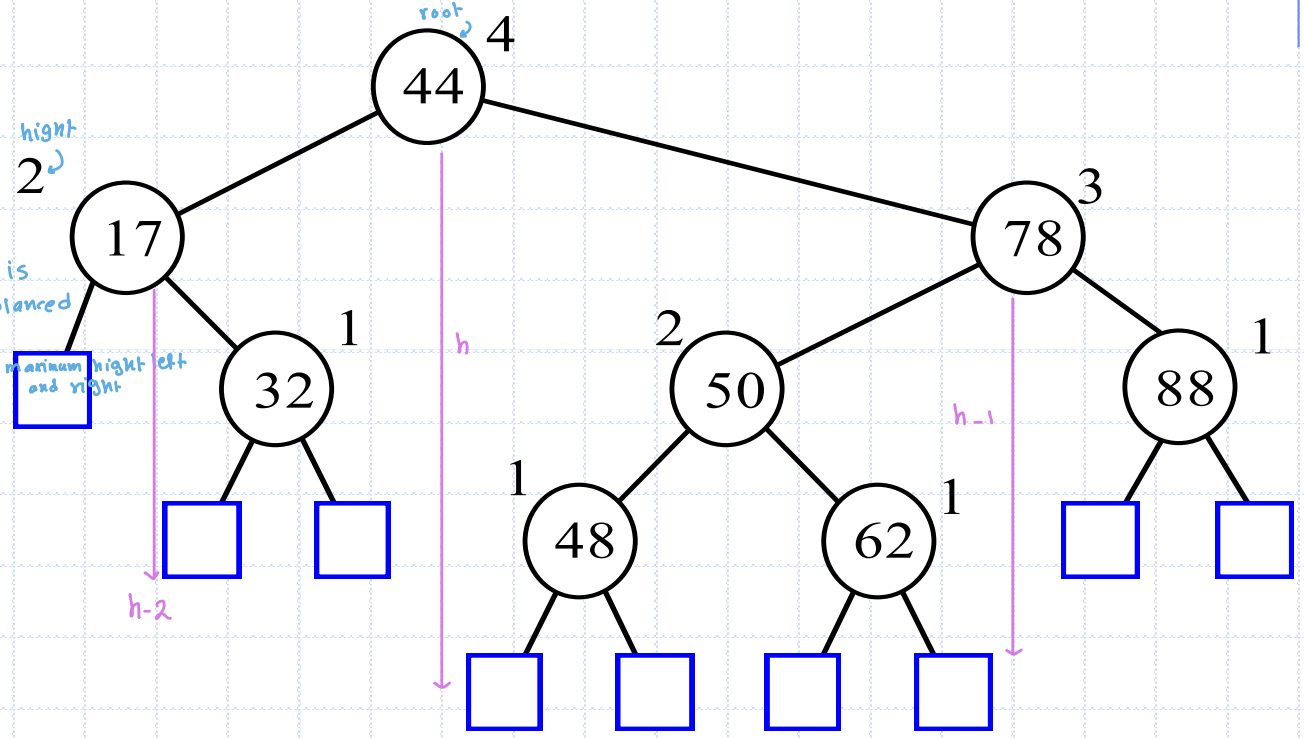
وفرق العلول منهما

يا : واحد او صفر او سالب واحد



AVL Tree Definition

- AVL trees are balanced
- An AVL Tree is a binary search tree such that for every internal node v of T, the heights of the children of v can differ by at most 1



An example of an AVL tree where the heights are shown next to the nodes

- Keys stored at nodes in the left subtree of v are less than or equal to k.
- Keys stored at nodes in the right subtree of v are greater than or equale to k.

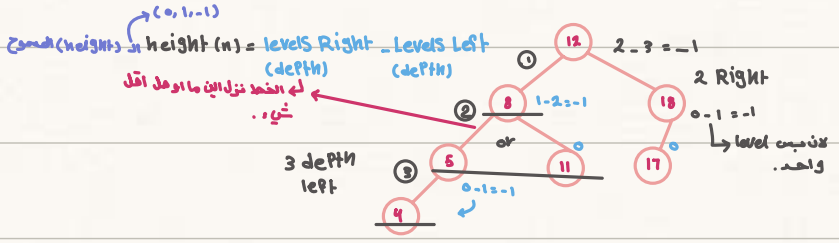
height AVL Tree

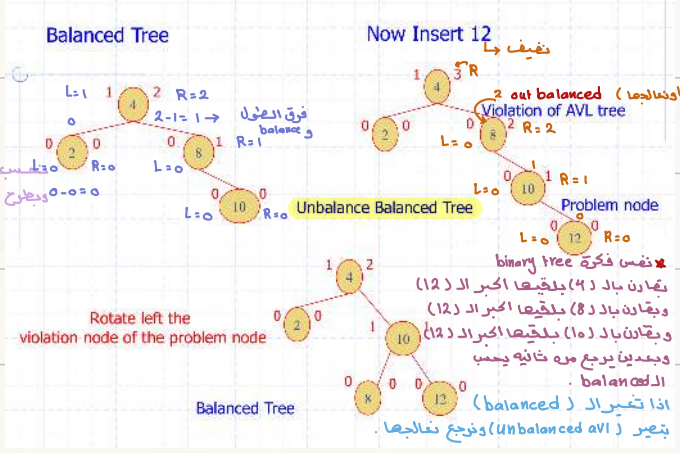
(left subtree) و (right subtree) الفرق بين العمق لـ

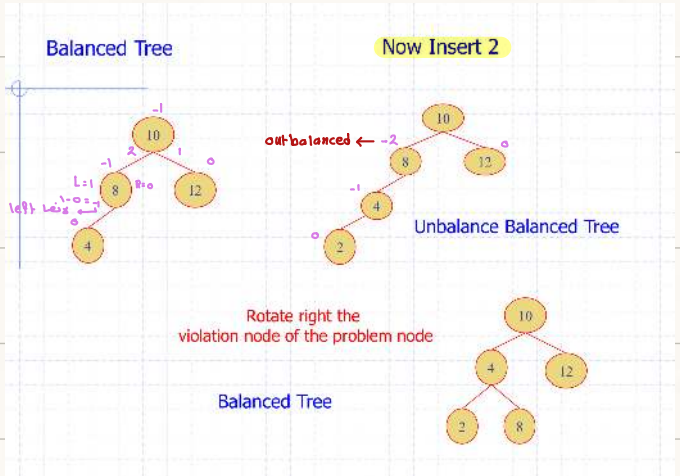
height(n) = levels Right (depth) - Levels Left (depth)

الصحيح (height) لـ (0, 1, -1)

لما نضيف نزل اللي ما فوقه اقل شيء.




Balanced Tree
Now Insert 12
Violation of AVL tree
Unbalance Balanced Tree
Problem node
Rotate left the
violation node of the problem node
Balanced Tree

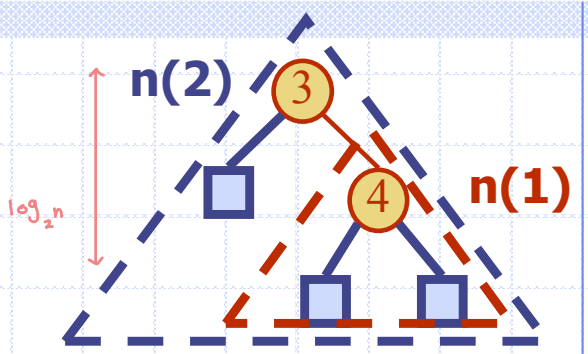

Balanced Tree
Now Insert 2
out-balanced
Unbalance Balanced Tree
Rotate right the
violation node of the problem node
Balanced Tree

the heights of the children

of v can differ by at most 1. (الفرق للعدل بين الاولاد مايكون اكثر من واحد)

balanced : متسمى

Height of an AVL Tree



Fact: The height of an AVL tree storing n keys is O(log n).

Proof (by induction): Let us bound n(h): the minimum number of internal nodes of an AVL tree of height h.

- We easily see that n(1) = 1 and n(2) = 2
- For n > 2, an AVL tree of height h contains the root node, one AVL subtree of height n-1 and another of height n-2.
- That is, n(h) = 1 + n(h-1) + n(h-2)
- Knowing n(h-1) > n(h-2), we get n(h) > 2n(h-2). So
  n(h) > 2n(h-2), n(h) > 4n(h-4), n(h) > 8n(n-6), … (by induction),
  $n(h) > 2^i n(h-2i)$
- Solving the base case we get: $n(h) > 2^{h/2-1}$
- Taking logarithms: h < 2log n(h) +2
- Thus the height of an AVL tree is O(log n)

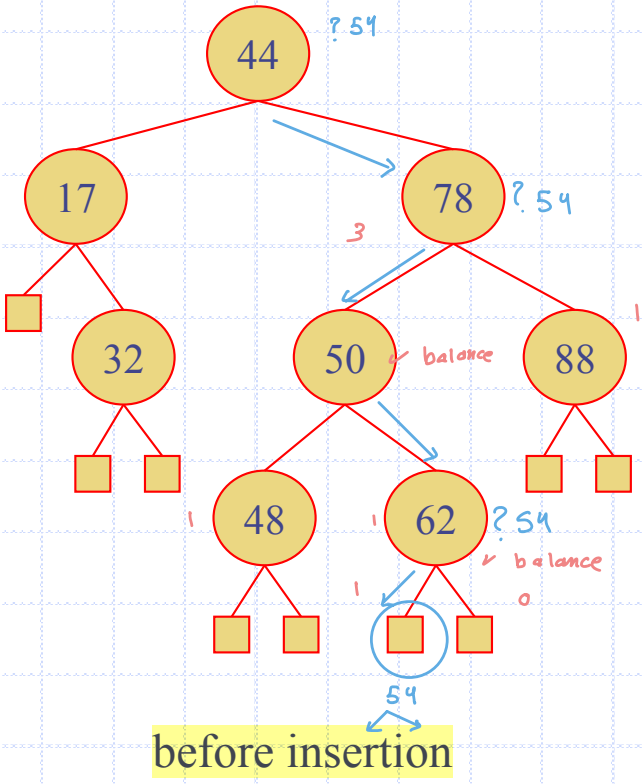
Beast casue: O(log n)

Worst casue: O(n)

Insertion

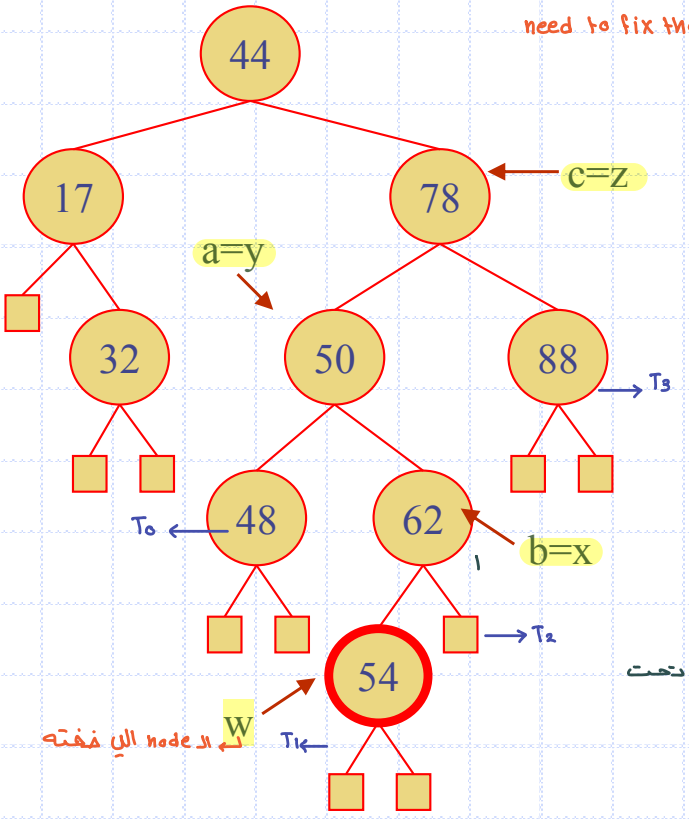
- Insertion is as in a binary search tree
- Always done by expanding an external node.
- Example: insert (54) root

height $> \log_2 n$



before insertion

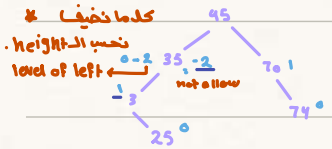
Height balanced of AVL tree is violated and need to fix that!



after insertion

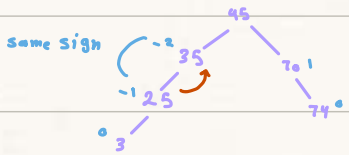
Creat an AVL tree by inserting the values:

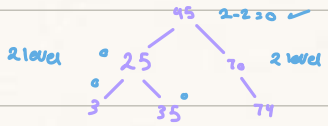
45, 70, 35, 3, 74, 25, 81, 60

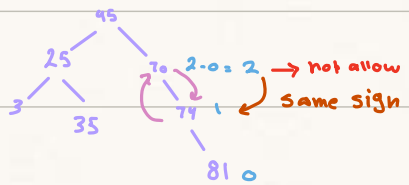


* كلما نخيف نحسب الـheight
level of left

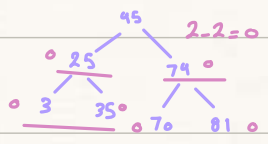
opposit sign (doble rotations)

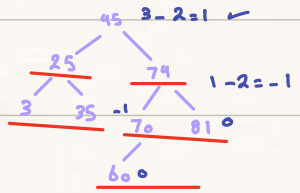






Single rotations





Complexity of Binary Search Tree

Operation	Average	Worst
Insert	O(log(n))	O(n)
Delete	O(log(n))	O(n)
Search	O(log(n))	O(n)

Complexity of Balanced Binary Search Tree → درخت متوازن

Operation	Average	Worst
Insert	O(log(n))	O(log(n))
Delete	O(log(n))	O(log(n))
Search	O(log(n))	O(log(n))

Trinode Restructuring

↳ working with 3 node.

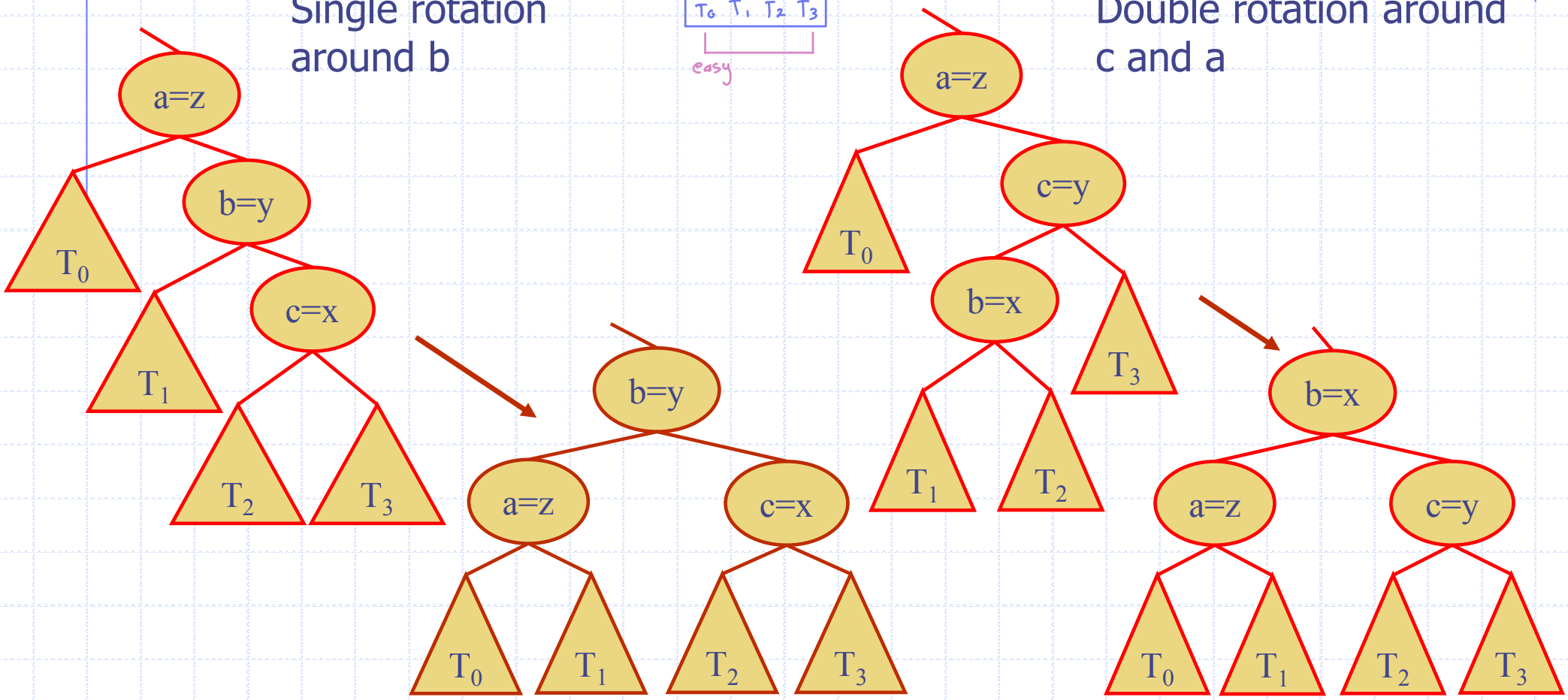
- Let (*a*,*b*,*c*) be the inorder listing of *x*, *y*, *z*
- Perform the rotations needed to make *b* the topmost node of the three

Single rotation around b

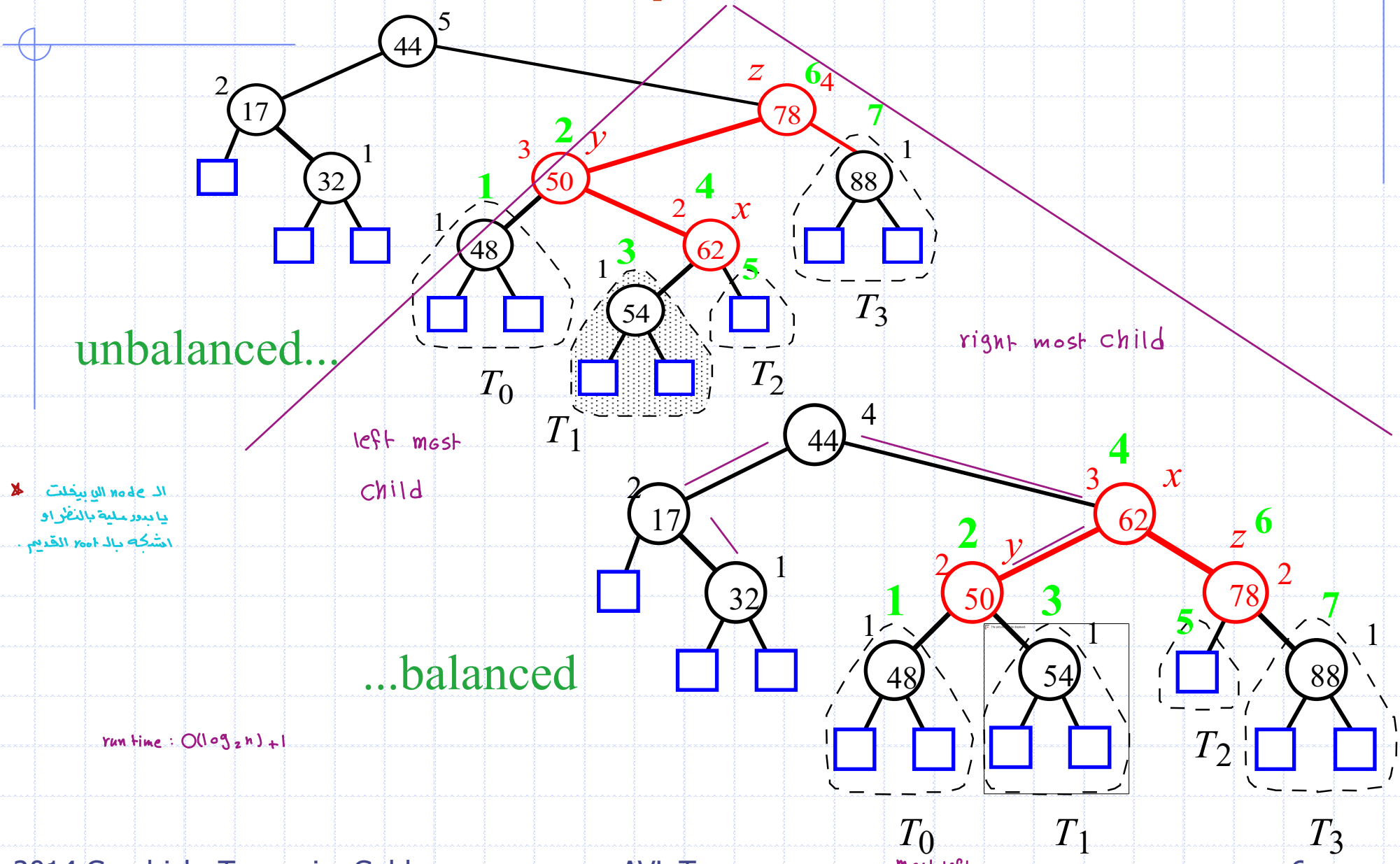
T_0 T_1 T_2 T_3

easy

Double rotation around c and a

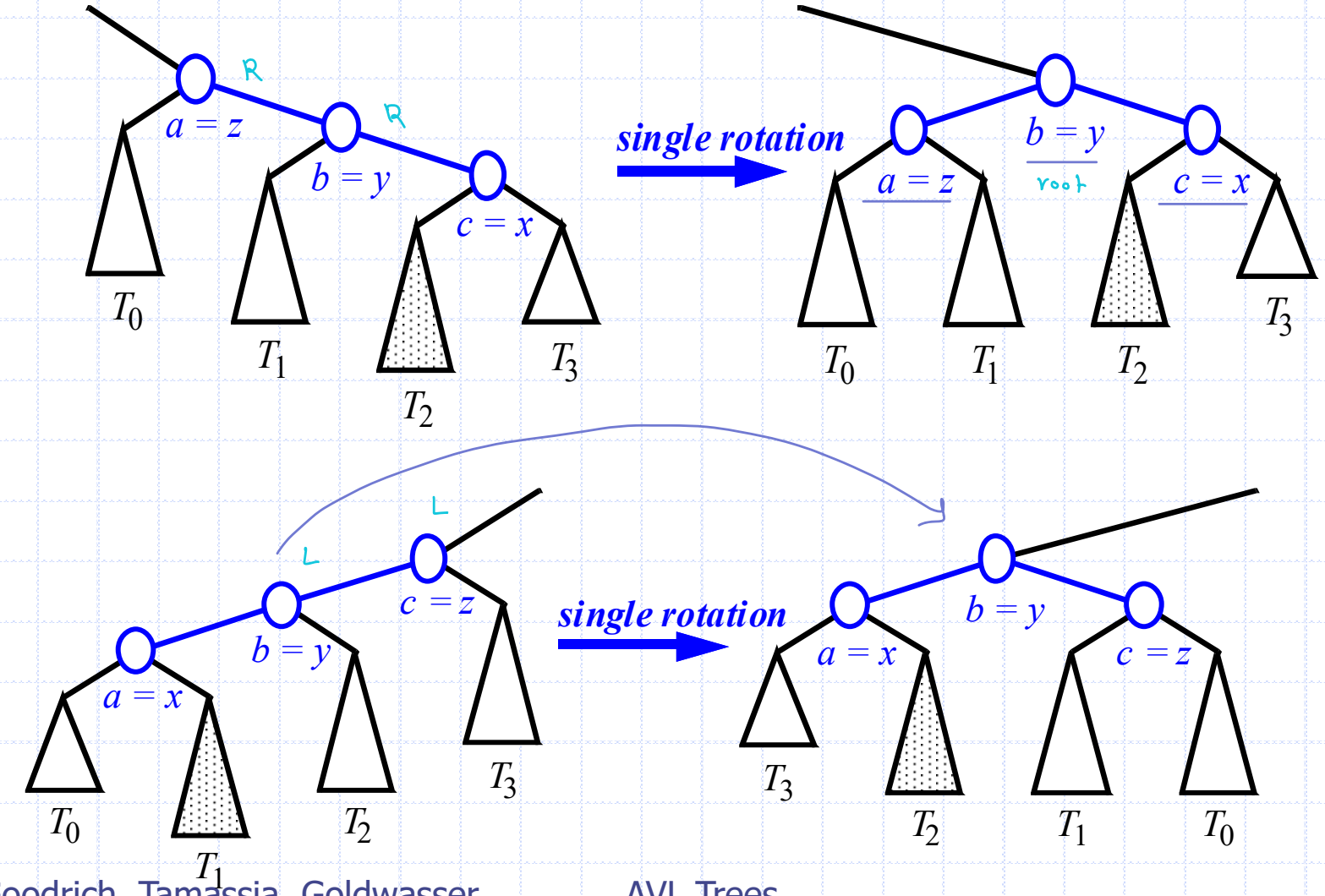


Insertion Example, continued



Restructuring (as Single Rotations)

- Single Rotations:



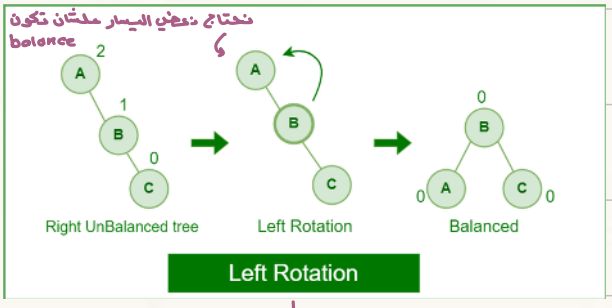
single

1- rotation (node مشكله) + (قبلها) same sign.

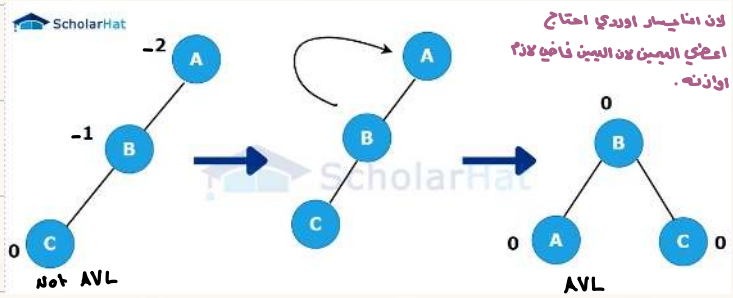
لم تدويره وحدة

↳ right الـ يكون نفسه والي قبله نفس الإشارة .

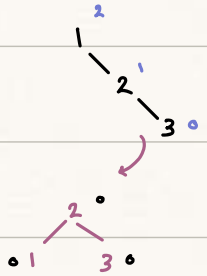
← الي قبل فيها المشكله تدفعها .



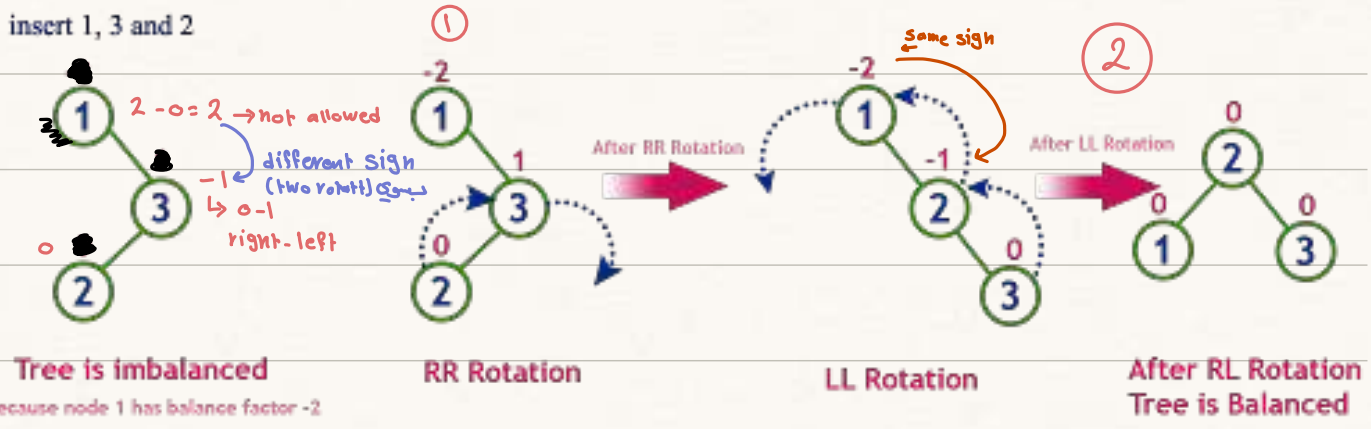
بنخلي الي قبل اليفية مشكله تدفعها لتحت .

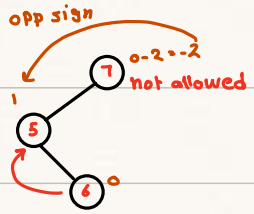


Ex :

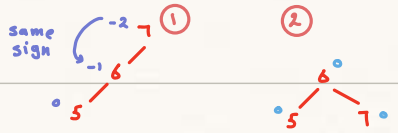


2- rotations (node مشكلة) + (قبلها) opp. sign
عكس ولا





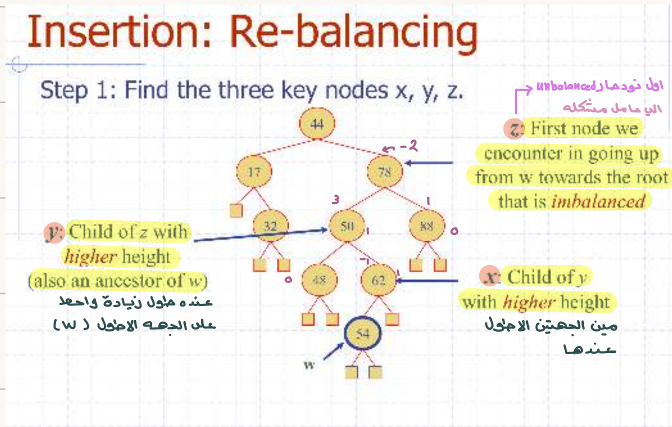
use double rotation



١- الي فيها مشكلة نشغب عليها ونركز على الثلاثة ونعاملها معاملة (one rotation)

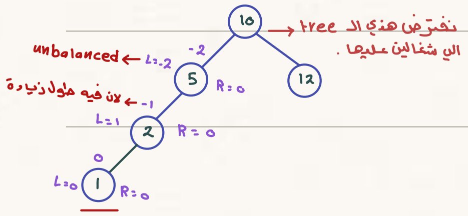
٢- معني مشكلة باليسار بدفعها يمين (الي تحت بتدفع الي فوق ، الي فوق بتنزل)

٣- الحين امسح الي شغبنه المشكلة واشوف .



✱ اول شيء حددنا الـ node الي عملية مشكله وسمينا الـ node الي بعدة .

كيف نعمل الـ balancing : لمن يصير عندي node ← unbalanced



1_ Insert element (1).

2_ نرجع نحسب الـ balance
بعد القيمة الي اضفتها .

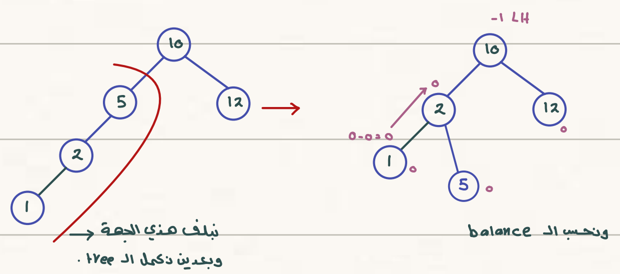
3_ حددنا الـ unbalanced
بعدة علشان نشوف ايش نوعها
لازم نحدد وين الطول الزايد اي جهة .

4_ حددنا الطول ← الطول الزيادة بالـ left
وال هو الاب (5) الي عامل مشكله .

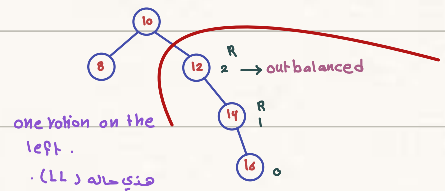
5_ اذا كانت المشكله في
LL or RR
نسوي (one rotion)

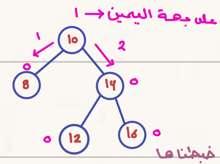
one rotion : عندها طول زيادة في الجهة
اليسار ، فا بنلفها بحيث نقلل من
الطول الزيادة .

فا بدال من الـ (5) هي الـ root بنزلها ونخلي الـ 2 هي الـ root .



نوع ثاني الـ mirror

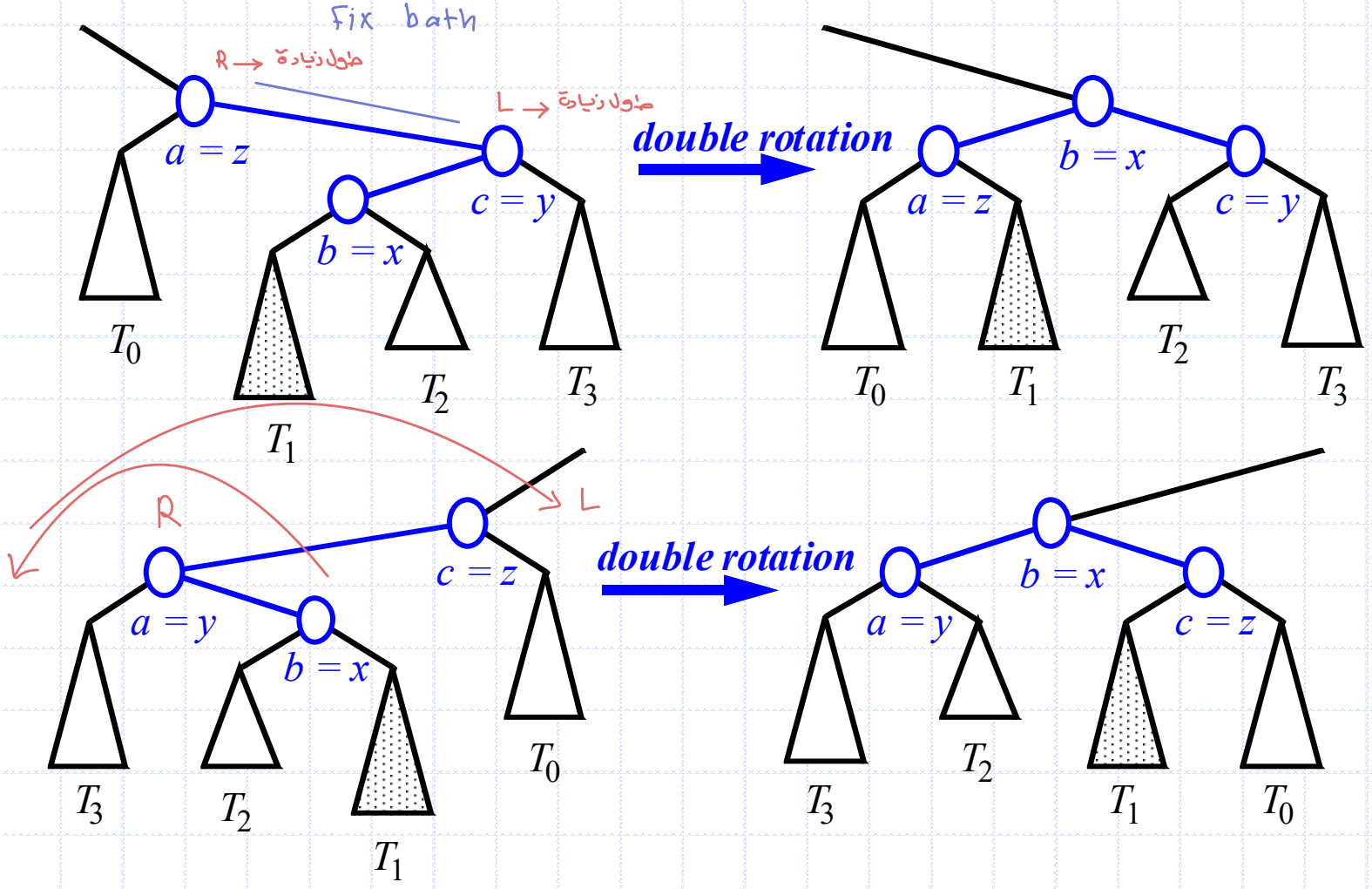




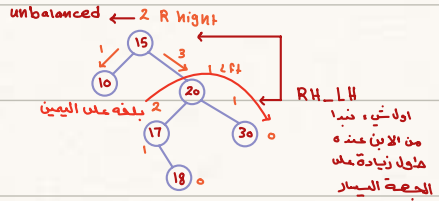
ونحسب الـ balanced بعد ما ضبطناها
هذي حاله (RR)

Restructuring (as Double Rotations)

- double rotations:

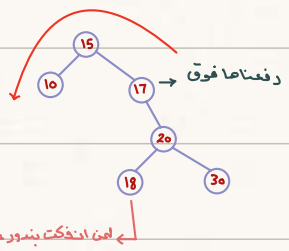


(as Double Rotations)

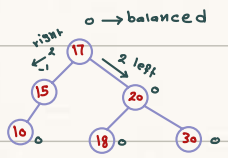


1- بنحسب الـ balance

2-

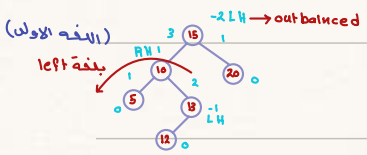


3- وبنلفها مره ثانية



عن امثلة : كان عندنا مشكلة
في الجهة الـ left ودخت
right فصارت right لفيتها
ورحنا لفيتها من حالة الابن left
والاب right بنلفها : للحاله الاولى (RR) .

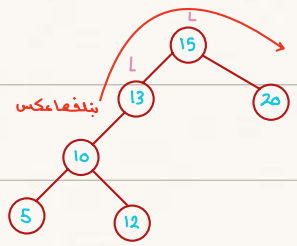
(LH_RH)



1- بنحسب الـ balanced

2- عندنا مشكلة بال left
بنروح لل left ومنشوف ايش
مشكلته بيطلع (right)

Double rotation : اذا اختلف النوعين معناته
دائما ابدا بالابن بعدين
الاب .

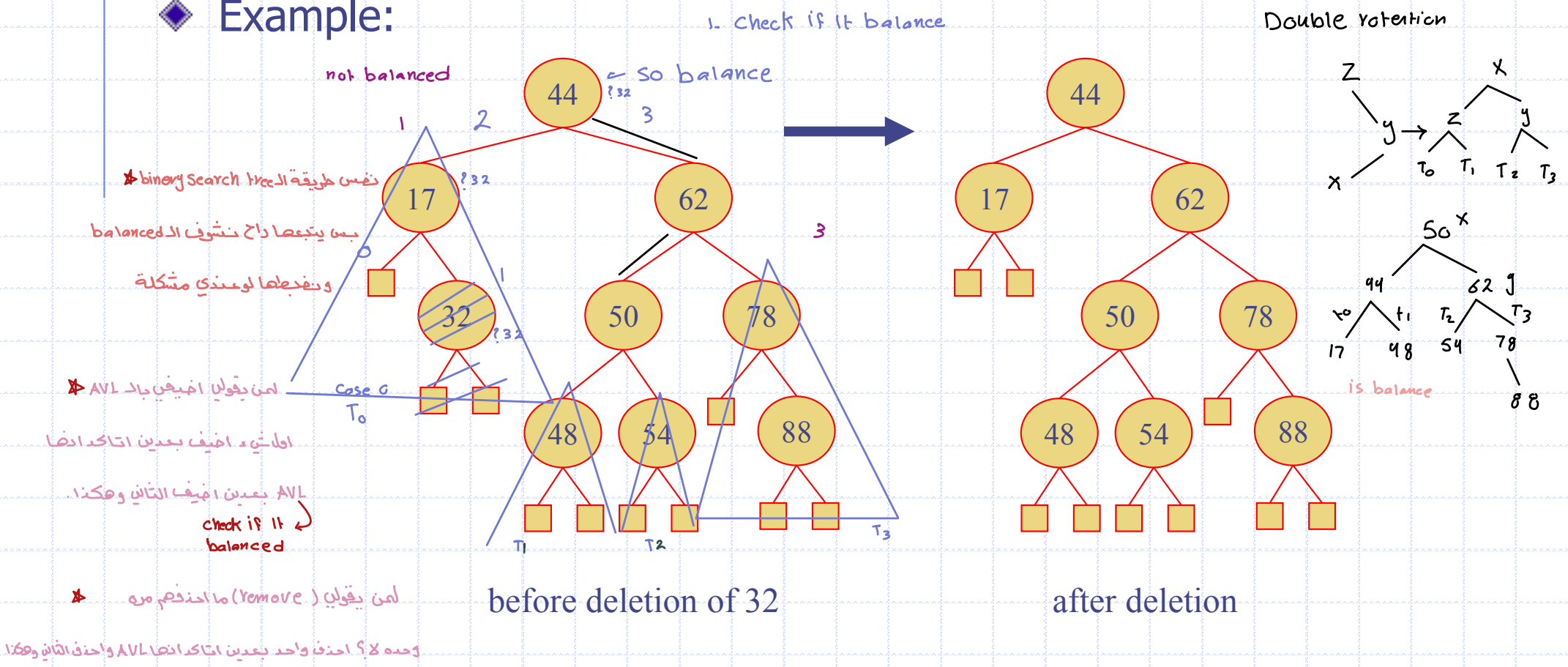


بعد ما خدمنا اللفه الاولى بنلفها اللفه الثانيه عكس →

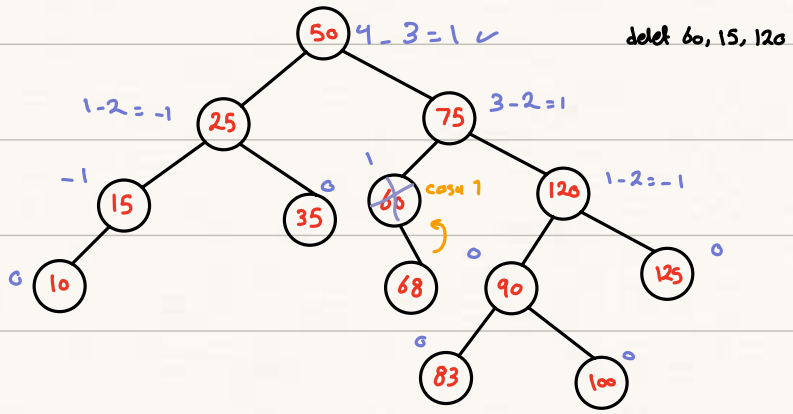
وبنحسب الـ balanced
بعد اللفه الثانيه.

Removal

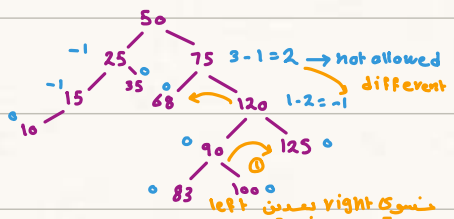
- Removal begins as in a binary search tree, which means the node removed will become an empty external node. Its parent, w, may cause an imbalance.
- Example:



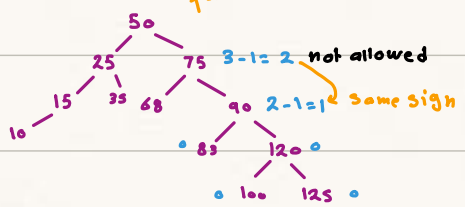
before deletion of 32 after deletion

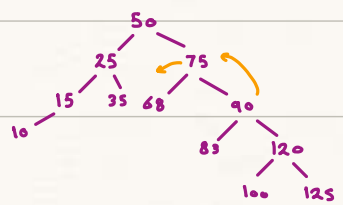


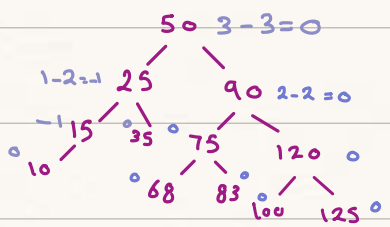
delet (60)



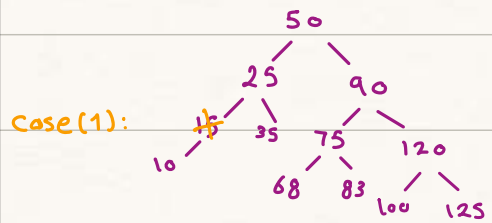
double rotation → لف يمين يمين يسار



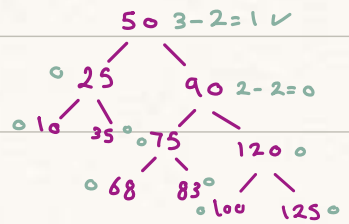




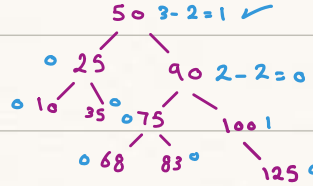
delet (15)



case (1):

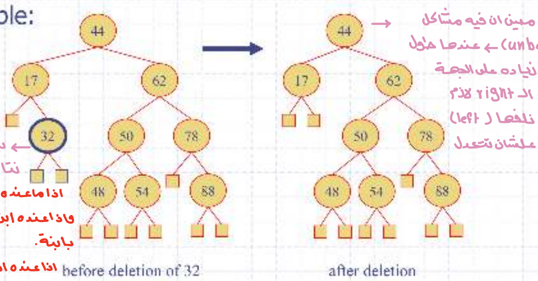


delet (12c)



Removal

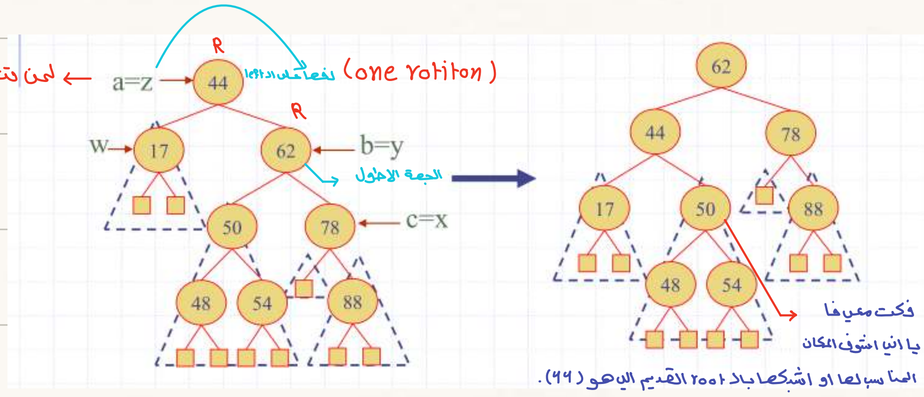
- Removal begins as in a binary search tree, which means the node removed will become an empty external node. Its parent, w, *may* cause an imbalance.
- Example:



مبين ان فيه مشاكل ← عندما طول (unbalanced) زياده على الجهة الـ right لازم نلخصا (left) علشان تتعدل

← بنحذفه وامو شبلازم نتأكد : + حذفناه لا ماعنده اولاد.

الـ (remove) العادية الي تعلمناها من الـ (tree). ←
1- اذا ماعنده اولاد بلغيه دايركت
2- واذا عنده ابن واحد بشبك ابوه بابنه.
3- اذا عنده ابنين عندي حلين يا الروح للجهة الـ (left) واخذ الـ node الكبير وابدله بالقيمه بعدين الغيه من تحت.
او اروح للجهة الـ (right) واخذ الـ node الصغيرة وابدله بالقيمه و بعدين الغيه من تحت.



لمن تتعدل ← a=z

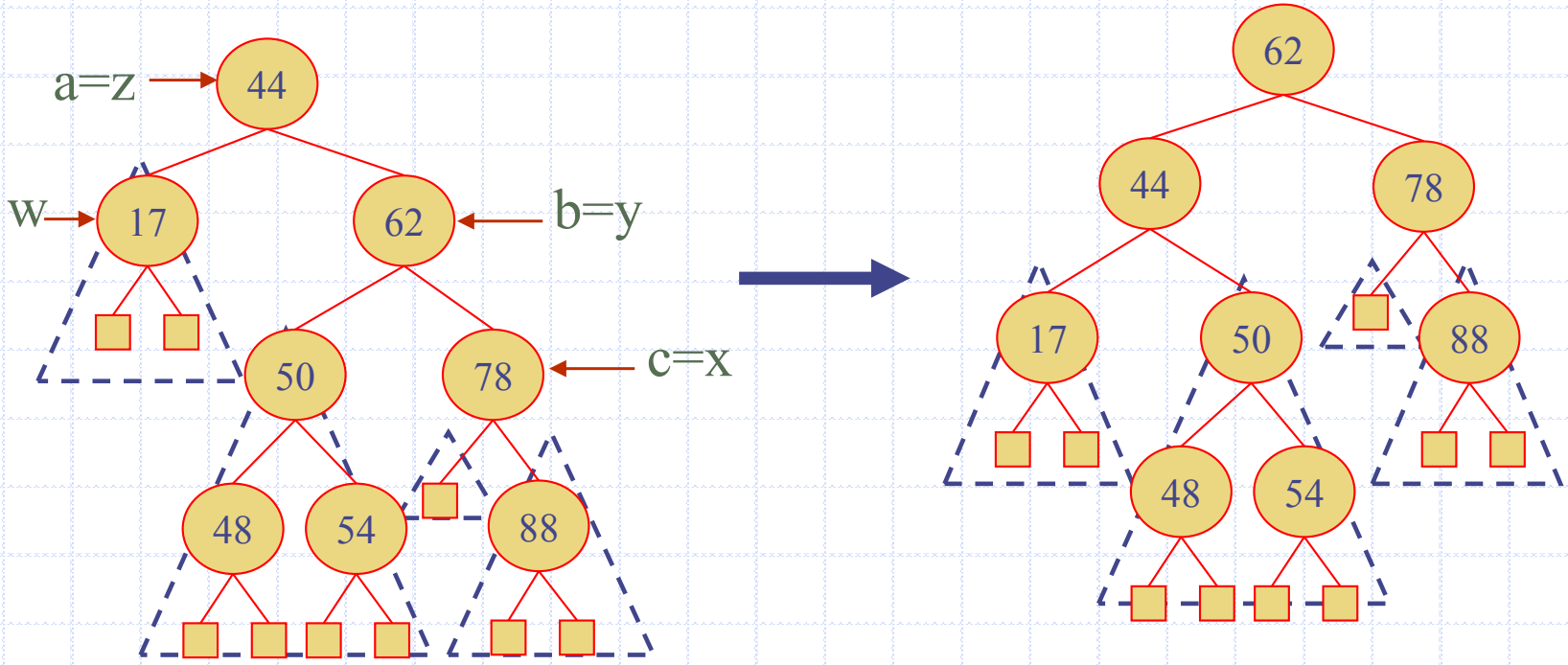
نفصل على الـ left (one rotiton)

الجهة الاطول

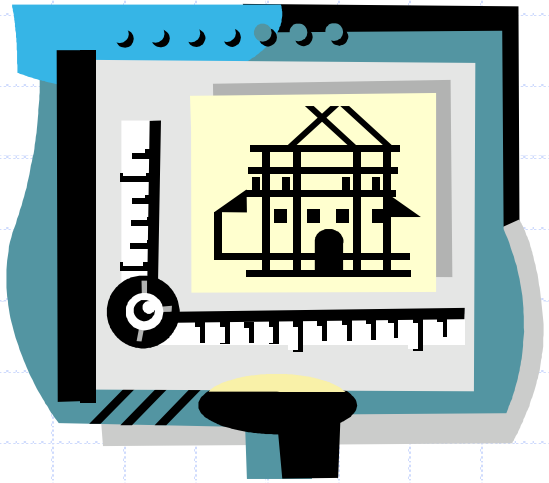
فكت معي ها يا اني اشوف المكان المناسب لها او اشبكها بالـ node القديم الي هو (٦٦).

Rebalancing after a Removal

- Let z be the first unbalanced node encountered while travelling up the tree from w. Also, let y be the child of z with the larger height, and let x be the child of y with the larger height
- We perform a trinode restructuring to restore balance at z
- As this restructuring may upset the balance of another node higher in the tree, we must continue checking for balance until the root of T is reached



AVL Tree Performance



- AVL tree storing n items
  - The data structure uses O(n) space
  - A single restructuring takes O(1) time
    - using a linked-structure binary tree → نرتبها او اضيف خانه او اشيل .
  - Searching takes O(log n) time → لانني بروح يمين ويسار
    - height of tree is O(log n), no restructures needed
  - Insertion takes O(log n) time → لان ببحث بالاول بعدين بضيف
    - initial find is O(log n)
    - restructuring up the tree, maintaining heights is O(log n)
  - Removal takes O(log n) time → بنبحث اول شيء بعدين بحذف وبعدله الـ tree .
    - initial find is O(log n)
    - restructuring up the tree, maintaining heights is O(log n)

Java Implementation

```
1  /** An implementation of a sorted map using an AVL tree. */
2  public class AVLTreeMap<K,V> extends TreeMap<K,V> {
3    /** Constructs an empty map using the natural ordering of keys. */
4    public AVLTreeMap( ) { super( ); }
5    /** Constructs an empty map using the given comparator to order keys. */
6    public AVLTreeMap(Comparator<K> comp) { super(comp); }
7    /** Returns the height of the given tree position. */
8    protected int height(Position<Entry<K,V>> p) {
9      return tree.getAux(p);
10   }
11   /** Recomputes the height of the given position based on its children's heights. */
12   protected void recomputeHeight(Position<Entry<K,V>> p) {
13     tree.setAux(p, 1 + Math.max(height(left(p)), height(right(p))));
14   }
15   /** Returns whether a position has balance factor between −1 and 1 inclusive. */
16   protected boolean isBalanced(Position<Entry<K,V>> p) {
17     return Math.abs(height(left(p)) − height(right(p))) <= 1;
18   }
```

Send a node

Java Implementation, 2

```
19    /** Returns a child of p with height no smaller than that of the other child. */
20    protected Position<Entry<K,V>> tallerChild(Position<Entry<K,V>> p) {
21      if (height(left(p)) > height(right(p))) return left(p);          // clear winner
22      if (height(left(p)) < height(right(p))) return right(p);         // clear winner
23      // equal height children; break tie while matching parent's orientation
24      if (isRoot(p)) return left(p);                                   // choice is irrelevant
25      if (p == left(parent(p))) return left(p);                        // return aligned child
26      else return right(p);
27    }
```

Java Implementation, 3

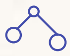
```
33    protected void rebalance(Position<Entry<K,V>> p) {
34      int oldHeight, newHeight;
35      do {
36        oldHeight = height(p);                    // not yet recalculated if internal
37        if (!isBalanced(p)) {                     // imbalance detected
38          // perform trinode restructuring, setting p to resulting root,
39          // and recompute new local heights after the restructuring
40          p = restructure(tallerChild(tallerChild(p)));
41          recomputeHeight(left(p));
42          recomputeHeight(right(p));
43        }
44        recomputeHeight(p);
45        newHeight = height(p);
46        p = parent(p);
47      } while (oldHeight != newHeight && p != null);
48    }
49    /** Overrides the TreeMap rebalancing hook that is called after an insertion. */
50    protected void rebalanceInsert(Position<Entry<K,V>> p) {
51      rebalance(p);
52    }
53    /** Overrides the TreeMap rebalancing hook that is called after a deletion. */
54    protected void rebalanceDelete(Position<Entry<K,V>> p) {
55      if (!isRoot(p))
56        rebalance(parent(p));
57    }
58  }
```

	AVL Tree		Binary Search Tree	
	Average	**Worst**	**Average**	**Worst**
Insertion	O(logn)	O(logn)	O(logn)	O(n)
Deletion	O(logn)	O(logn)	O(logn)	O(n)
Searching	O(logn)	O(logn)	O(logn)	O(n)
Traversal	O(n)	O(n)	O(n)	O(n)

BT_BSt_AVL Rules :

اول شيء اخذنا الـ BT بعدين الـ BST بعدين الـ AVL

1_ BT → maximum child (2)



تطورت وصارت

2_ BSt → maximum احلها ثنائيه باد
عندها ترتيب بالذات
الاب اصغر من الـ left
واكبر من الـ right

مستحيل يصير غير هالقوانين ، لازم الـ rule
تنطابق على كل الـ node اذا node واحد خرب الفكرة
معناته : بطل هذا النوع صار النوع الي اقل منه .

3_ AVL → binary search tree عباره عن
ان الـ (left) اصغر والـ (right) الكبر
وقاعدتها الاساسية : ان هذي الـ tree سوء اخفنا او حذفنا
بنحطها باصغر (level) ممكنة
يكون الفرق الـ (hight) بين الاولاد اكثر
من واحد . اذا كانت اكثر من واحد بطلت
(AVL tree) بتكون (BST) .

بتكون ← balanced tree

maximum (BT) :
ماعندها قاعدة لازم
البحث بال tree كامله

maximum (BST) : orderd data
بيكون باقصى اليمين →

```
while (P.right != null)
    P = P.right
    return P.data
```

minum (BST) :
بيكون باقصى اليسار

```
          root
While (P.left != null)
P = P.left
      return P.data
                or key
```

← خلك تحرك (P = P.left)

remove :

(BT) = remob sub ← بنسوي بلغي الـ node والي تحته

(BST) :
Same rule — 3 causes
(AVL) :

1) no child : بلغيه
2) on child : بشبك الابن بالاب
3) two child : يا باخذ الـ min من اقصى اليمين بلغيه واحطه فوق .

(AVL) ← يتبعها ترتيب الـ tree
و نسوي balance .

Insert :

(BS): انا احمد وين اضيف لان
ما فيه قاعدة امشي عليها

(BST):
(AVL): — Same rule

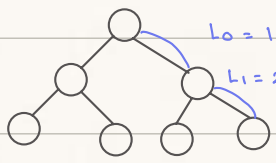
الإضافة حسب القيمة الـ key
نعمل search بعدين نقدر وين
نضيف بعدين insert.

Search: بنشوف وين المكان
المناسب ونضيفه. لكن الـ (AVL)
نحتاج لتعديل (balanced).

---

hight :

h = 2



$L_0 = 1 \rightarrow 2^0 = 1$

$L_1 = 2 \rightarrow 2^1 = 2$

$L_2 = 4 \rightarrow 2^2 = 4$

عدد النودات في نفس level ← 2^{level}

عدد النودات كامله بال tree = $1+2+4 = 7$

$2^{h+1} - 1 = 2^3 - 1 = 8 - 1 = 7$

max = $2^{h+1} - 1 = 2^3 - 1 = 7$

min = $h + 1 = 3$

نحسبه من الـ edge